Task 1: Code and analyze solutions to following problem with given strategies:

1.i. Knap Sack using greedy approach.

```
#include <iostream>
#include <algorithm>
using namespace std;
struct Item {
    int weight;
    int profit;     };
bool compare(Item a, Item b) {
    double r1 = (double)a.profit / a.weight;
    double r2 = (double)b.profit / b.weight;
    return r1 > r2;    }

int main() {
    int n;
    cout << "Enter number of items: ";
    cin >> n;
    Item items[n];
    for (int i = 0; i < n; i++) {
        cout << "Enter weight and profit of item " << i + 1 << ": ";
        cin >> items[i].weight >> items[i].profit;
    }
    int capacity;
```

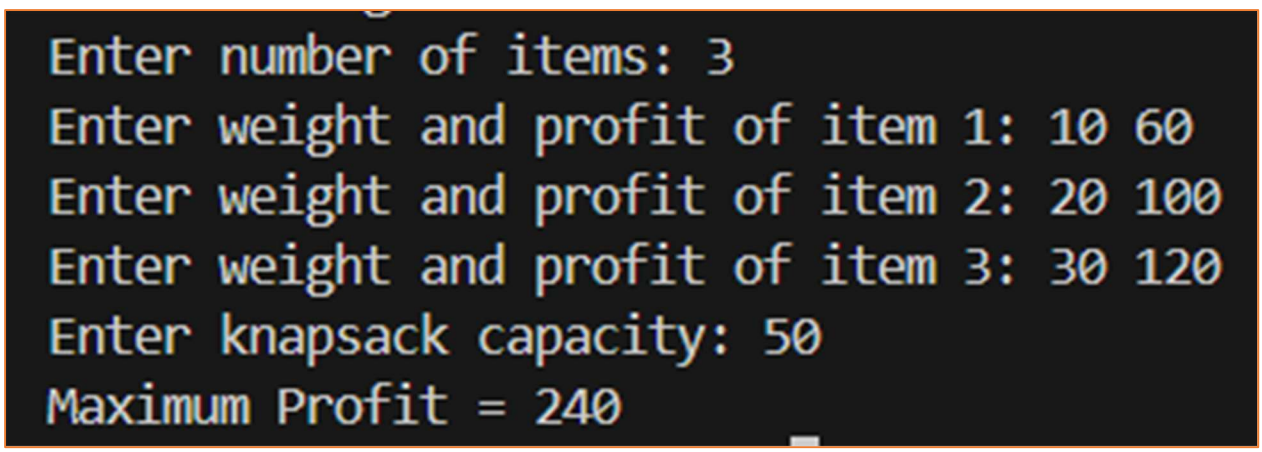
```
    cout << "Enter knapsack capacity: ";
    cin >> capacity;

    sort(items, items + n, compare);
    double totalProfit = 0.0;
    for (int i = 0; i < n; i++) {
        if (capacity >= items[i].weight) {
            capacity -= items[i].weight;
            totalProfit += items[i].profit;
        } else {
            totalProfit += (double)items[i].profit * capacity / items[i].weight;
            break;
        }
    }

    cout << "Maximum Profit = " << totalProfit << endl;

    return 0;
}
```

OUTPUT:-



Task 1: Code and analyze solutions to following problem with given strategies:

1.ii. Knap Sack using dynamic approach.

```
#include <iostream>
using namespace std;
int max(int a, int b) {
   return (a > b) ? a : b;
}
int main() {
   int n, W;
   cout << "Enter number of items: ";
   cin >> n;
   int weight[n], profit[n];

   for (int i = 0; i < n; i++) {
      cout << "Enter weight and profit of item " << i + 1 << ": ";
      cin >> weight[i] >> profit[i];
   }
   cout << "Enter knapsack capacity: ";
   cin >> W;
   int dp[n + 1][W + 1];

   for (int i = 0; i <= n; i++) {
      for (int w = 0; w <= W; w++) {
```

```
            if (i == 0 || w == 0)
                dp[i][w] = 0;

            else if (weight[i - 1] <= w)
                dp[i][w] = max(
                    profit[i - 1] + dp[i - 1][w - weight[i - 1]],
                    dp[i - 1][w]
                );

            else
                dp[i][w] = dp[i - 1][w];
        }
    }
    cout << "Maximum Profit = " << dp[n][W] << endl;
    return 0;
}
```

OUTPUT:-